Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Structures de données séquentielles et gestion de la mémoire

Synthèse exécutive : Ce cours aborde la manipulation des tableaux à une dimension en langage C, la contiguïté mémoire et le lien fondamental entre tableaux et pointeurs. L'objectif est de comprendre comment structurer une suite de données de même type en mémoire, comment y accéder de manière efficace via des indices ou des pointeurs, et comment transmettre ces tableaux à des fonctions sans duplication mémoire inutile.

1. Définition et représentation mémoire d'un tableau

En langage C, un tableau est une plage d'adresses mémoire consécutives réservées pour stocker un nombre fixe d'éléments du même type. Lorsque vous déclarez `int tab[5];`, le compilateur réserve un bloc continu de `5 * sizeof(int)` octets. L'accès à un élément via `tab[i]` s'effectue en temps constant O(1) grâce à un calcul d'adresse directe : `Adresse(tab[i]) = Adresse_Base + (i * Taille_Type)`. C'est la raison pour laquelle le premier indice est 0 : son décalage (offset) par rapport au début du tableau est de zéro.

- Tableau : Conteneur de taille fixe stockant des éléments de type identique.
- Contiguïté mémoire : Les cases mémoire sont exactement côte à côte sans espace intermédiaire.
- Indice : Position d'un élément, débutant obligatoirement à 0.

Cas concret enseignant : *Le professeur a comparé le tableau à un meuble de rangement à tiroirs identiques numérotés de 0 à N-1 collés les uns aux autres. Si l'on connaît l'emplacement du premier tiroir et la hauteur exacte d'un tiroir, on peut atteindre n'importe quel tiroir immédiatement sans les ouvrir un par un.*

2. Alignement, pointeurs et dégradation d'un tableau (Array Decay)

Lorsqu'un tableau est transmis en paramètre à une fonction ou utilisé dans une expression, le nom du tableau s'évalue automatiquement vers l'adresse mémoire de son premier élément (`&tab;[0]`). Ce phénomène est appelé 'Array Decay'. Par conséquent, transmettre un tableau à une fonction ne copie pas l'intégralité des données en mémoire (passage par valeur désactivé pour la performance), mais transmet simplement une adresse mémoire (passage par adresse/pointeur). Il est donc absolument nécessaire de passer la taille du tableau en paramètre séparé à la fonction.

- Array Decay : Conversion implicite du nom d'un tableau en un pointeur vers son premier élément.
- Arithmétique des pointeurs : `*(tab + i)` est strictement équivalent à `tab[i]`.
- Perte de taille : Une fonction recevant un tableau ne connaît plus sa taille automatiquement.

Cas concret enseignant : *Pour illustrer cela, l'enseignant a expliqué que donner un tableau à une fonction, c'est comme donner l'adresse GPS de sa maison au lieu d'en construire une réplique exacte dans le jardin de son ami. La fonction agit directement sur l'original.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le code plante ou affiche des valeurs absurdes quand j'essaye de lire `tab[5]` alors que le tableau a une taille de 5 ?
Réponse du professeur : Un tableau de taille 5 possède ses éléments aux indices 0, 1, 2, 3 et 4. L'indice 5 pointe en dehors de la mémoire allouée (Dépassement de tableau / Buffer Overflow). En C, il n'y a pas de contrôle automatique des bornes à l'exécution : vous lisez une case mémoire qui appartient potentiellement à une autre variable, provoquant un comportement indéterminé ou une Erreur de Segmentation (Segmentation Fault).

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau dans une fonction avec `sizeof(tab) / sizeof(tab[0])` ?
Réponse du professeur : Non ! Dans la fonction reçue, le paramètre `tab` s'est dégradé en simple pointeur (`int*`). La commande `sizeof(tab)` renverra donc la taille d'un pointeur (4 ou 8 octets) et non la taille globale du tableau. Cette astuce ne fonctionne que dans la portée exacte où le tableau a été déclaré.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C `remplir_et_calculer_moyenne` qui prend en paramètre un tableau d'entiers, sa taille, le remplit avec des notes saisies par l'utilisateur, puis retourne la moyenne arithmétique sous forme de nombre à virgule (`double`).

Résolution pas à pas :
1. Définir le prototype : `double calculer_moyenne(int tab[], int taille);`. 2. Initialiser une variable accumulateur `double somme = 0.0;`. 3. Parcourir le tableau de `i = 0` à `i < taille` à l'aide d'une boucle `for`. 4. Demander la saisie de chaque élément avec `scanf("%d", &tab;[i]);` et l'ajouter à `somme`. 5. Une fois la boucle terminée, diviser `somme` par `taille` (en s'assurant que `taille > 0`). 6. Retourner le résultat.

Résultat : #include double calculer_moyenne(int tab[], int taille) { if (taille <= 0) return 0.0; double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Note %d : ", i + 1); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple de résultat pour les notes 12, 14, 16 : Moyenne = 14.00

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément d'un tableau déclaré avec `int t[10];` ?

R : L'indice du dernier élément est 9, car le premier élément commence à l'indice 0.

Q : Que contient le nom d'un tableau lorsqu'il est utilisé sans crochets ?

R : Il contient l'adresse mémoire du tout premier élément du tableau (`&t;[0]`).

Q : Pourquoi est-il obligatoire de passer la taille d'un tableau en paramètre d'une fonction ?

R : Parce qu'à la transmission, le tableau est converti en simple pointeur (Array Decay). La fonction n'a plus aucun moyen de connaître la taille de la zone mémoire allouée.